

SBA301 – SLOT 10

Chapter 09: React Router and Single Page Application (SPA)

React Router Demo • Client-side Routing • URL State • Nested Routes • Lab 02 Kick-off

Môn: SBA301 – Integrate Single Page Application with Spring Boot

CLO2 – Apply fetching/caching data in Client–Server Communication

Phạm vi: Chapter 09, slide 1–22 | Không đi sâu Fetch/Axios hoặc Spring Boot backend

Student Learning Guide mở rộng • 35–45 trang mục tiêu • Project-based Learning

Mental model xuyên suốt: URL → Router → route matching → layout → page component → UI. Navigation thay đổi URL/history và React tree mà không cần tải lại toàn bộ document.

Engineering rule: Không đánh giá routing bằng “click được menu”. Sinh viên phải chứng minh URL, active navigation, history Back/Forward, dynamic params, nested rendering, direct URL, 404 và route-state ownership.

CÁCH SỬ DỤNG GIÁO TRÌNH

Tài liệu được thiết kế như Student Learning Guide, không chỉ là slide handout. Mỗi chủ đề đi qua 6 bước: Concept → Mental model → Worked example → Trace/DevTools → Break-It → Transfer sang Lab 02/Assignment.

Pha	Sinh viên thực hiện	Bằng chứng
Prepare	Ôn Slot 9, chạy React project, kiểm tra Node/npm.	Pre-check + baseline screenshot
Model	Vẽ URL → Router → Route → Component.	Route tree/trace worksheet
Build	Cấu hình BrowserRouter, Routes, Route, Link/NavLink.	App chạy + URL đúng
Navigate	Dùng params, query, location, navigate, nested routes.	History/route evidence
Break	Tạo lỗi missing Outlet, wrong param, <a>, wildcard, direct URL.	Break-It log
Transfer	Chuyển Orchid Lab 02 từ component app sang SPA có route.	Lab 02 bridge evidence
Verify	Human Gate + Git evidence + reflection.	E01–E14

AI rule: AI may generate route code; student must run it, explain match/URL/history flow, verify behavior with DevTools và tự sửa ít nhất một routing bug.

0. SINH VIÊN CẦN CHUẨN BỊ GÌ TRƯỚC KHI HỌC BUỔI NÀY

0.1 Ôn lại từ Slot 9 – Fetching and Caching Data

- Promise và async/await: hiểu rằng data/API flow là bất đồng bộ.
- Fetch/Axios và service layer: biết dữ liệu có thể được tải theo từng page/route.
- Loading/Error/Data state: route không thay thế data state; hai lớp trách nhiệm khác nhau.
- useEffect: hiểu side effect và cleanup cơ bản.
- HTTP/REST: URL frontend route và API endpoint là hai không gian URL khác nhau.

0.2 Kiến thức React nền tảng cần nhớ

- Function component, props, state, useState, useEffect.
- Component composition: Layout chứa Navbar và Outlet/Page.
- React Bootstrap: Container, Navbar, Nav, Card, Button, Row, Col.
- Array.map và key khi render danh sách.

0.3 Công cụ

Công cụ	Mục đích	Checklist
JDK	Không dùng trực tiếp trong Slot 10	Chuẩn bị cho Chapter 10 về sau
Node/npm	Chạy Vite/React	node -v, npm -v
IntelliJ IDEA / VS Code	IDE	Terminal mở được
Browser DevTools	History/Network/Console	Console sạch
Git	Evidence & checkpoint	git status sạch trước slot

0.4 Cập nhật tương thích 2026 – rất quan trọng

Course compatibility track: Nếu phòng lab giữ Node 21, không dùng `npm create vite@latest`. Pin Vite 5 và React Router v7.18 để giữ cú pháp `react-router-dom` quen thuộc trong giáo trình.

```
# Node 21 / course-compatible track
npm create vite@5 orchid-router-demo -- --template react
cd orchid-router-demo
npm install
npm install react-router-dom@7.18.0 react-bootstrap bootstrap
```

Current ecosystem note (20/09/2026): React Router v8.4.0 hiện hành đã nâng baseline lên Node 22+, React 19+ và bỏ `react-router-dom` re-export package; declarative APIs vẫn giữ tinh thần BrowserRouter/Routes/Route nhưng import hiện đại đi từ `react-router`. Nội dung cốt lõi của buổi học vì vậy vẫn transferable.

0.5 Pre-class 3–2–1

1. 3 điều em nhớ về URL/history/navigation.
2. 2 điểm khác nhau em dự đoán giữa SPA và MPA.
3. 1 câu hỏi về React Router hoặc direct URL refresh.

0.6 Pre-check 8 câu

1. Component khác page như thế nào?
2. State của component có tồn tại sau full page reload không?
3. `` làm browser làm gì?
4. URL path và API endpoint có nhất thiết giống nhau không?
5. useState dùng cho navigation history không?
6. DevTools có thể quan sát URL/history ở đâu?
7. Nếu `/orchids/3` được refresh trực tiếp, web server cần làm gì?
8. Tại sao route 404 khác API 404?

I. TÓM TẮT CÁC CHỦ ĐỀ & MỤC TIÊU TRONG BÀI HỌC

1. Tóm tắt kiến thức Slot 9 và khoảng trống Slot 10

Chủ đề Slot 9	Đã biết	Khoảng trống Slot 10 giải quyết
Promise / async-await	Xử lý bất đồng bộ	Điều hướng app mà không reload document
Fetch / Axios	Gọi API	Tổ chức UI thành route/page theo URL
Loading/Error/Data	Quản lý data state	Quản lý navigation/location state
Caching	Tối ưu dữ liệu	Giữ URL có nghĩa, bookmark/share/back-forward
Service layer	Tách HTTP khỏi UI	Tách routing/layout khỏi feature component

2. Định vị Slot 10 trong roadmap

Slot	Chủ đề	Quan hệ với Slot 10
8	HTTP, JSON, REST	URL/API foundations
9	Fetching & Caching	Data vào UI
10	React Router & SPA	URL → UI composition
11	Lab 02 + Assignment 01	Thực hành tích hợp component/navigation
12	REST Fundamentals	Frontend route bắt đầu kết nối backend resource

3. Bối cảnh bài học

Một ứng dụng React có thể render nhiều component nhưng vẫn chỉ ở một URL. Khi app phát triển thành Home, Orchids, Orchid Detail, About, Dashboard..., người dùng cần URL có ý nghĩa, nút Back/Forward đúng, bookmark/share được và điều hướng không tải lại toàn bộ HTML. React Router cung cấp lớp ánh xạ giữa location của browser và React component tree.

4. Mục tiêu học tập

- Giải thích SPA/MPA và client-side/server-side routing.
- Cài đặt Router theo cấu hình tương thích khóa học.
- Xây route tree bằng BrowserRouter, Routes, Route.
- Điều hướng bằng Link/NavLink và programmatic navigation.
- Đọc route params, query params và location state.
- Tổ chức nested route/layout bằng Outlet và index route.
- Thiết kế 404/redirect/history/direct URL behavior.
- Hoàn thành React Router demo và bắt đầu Lab 02 Orchid SPA.

5. Khái niệm trung tâm

Khái niệm	Câu hỏi phải trả lời
SPA	Document HTML có reload khi chuyển view không?
Location	URL hiện tại gồm pathname/search/hash/state gì?
Route	Pattern URL nào map vào element nào?

Khái niệm	Câu hỏi phải trả lời
Navigation	Ai làm URL/history thay đổi?
Params	Phần biến đổi nào nằm trong path?
Query	Filter/sort/search nào nên shareable trong URL?
Outlet	Child route render ở đâu trong parent layout?
History	Back/Forward/replace hoạt động thế nào?

6. Flow tổng quát

Flow: User click NavLink → History API cập nhật location → Router match route tree → Layout giữ nguyên → Outlet/Page thay đổi → component render → nếu cần page tự lấy data.

7. Tình huống dẫn nhập – FUNewsManagementSystem

Menu Dashboard, Category, News, Users, Settings không nên dùng 5 HTML document độc lập. Route tree giúp mỗi màn hình có URL riêng, active menu đúng, Back/Forward hoạt động và page có thể tải data riêng.

8. Tư duy kiến trúc

- URL là một phần của application state, không chỉ là text trên address bar.
- Route hierarchy nên phản ánh information architecture.
- Layout và page có vòng đời/trách nhiệm khác nhau.
- Navigation state không nên bị trộn với server data state.
- Direct URL và refresh phải được thiết kế, không chỉ click nội bộ.

9. Kết luận phần

Learning checkpoint: Nếu sinh viên chỉ “biết thêm Route” nhưng chưa giải thích được URL → route match → layout → page → history, thì chưa đạt mục tiêu Slot 10.

II. TRỌNG TÂM CỐT LÕI & KỸ NĂNG SINH VIÊN CẦN NẮM VỮNG

1. Single Page Application (SPA) là gì?

SPA là ứng dụng web có một document shell chính. Khi navigation xảy ra, JavaScript thay đổi URL/history và render view mới trong shell thay vì yêu cầu browser tải một HTML document hoàn toàn mới. SPA không có nghĩa là “chỉ có một màn hình”, mà là nhiều views/pages được tổ chức phía client.

1.1 SPA không đồng nghĩa client-only forever

SPA có thể vẫn dùng backend REST, CDN, SSR/hydration hoặc pre-render ở production. Trong Slot 10, ta học client-side routing ở mức React SPA để nắm nền tảng.

2. SPA vs MPA – so sánh có điều kiện

Tiêu chí	SPA	MPA / server navigation
Navigation	Thường không reload document	Request document mới
Initial load	Có thể tải JS bundle đáng kể	HTML page có thể nhẹ hơn
Subsequent navigation	Nhanh nếu bundle/data sẵn	Phụ thuộc server/network
URL/history	Router phía client phối hợp History API	Browser/server quản lý trực tiếp
SEO	Cần kiến trúc/rendering phù hợp	HTML server thường dễ crawl hơn
Complexity	State/routing/build/deploy phức tạp hơn	Server templates/routing có complexity khác
Data	Thường gọi APIs	Có thể render data trong HTML

Không tuyệt đối hóa: Không nên dạy “SPA luôn nhanh/MPA luôn chậm” hoặc “SPA SEO kém”. Kết quả phụ thuộc rendering strategy, caching, bundle, server, network và product architecture.

3. Client-side routing vs server-side routing

Client-side routing xử lý location và lựa chọn component trong runtime của frontend. Server-side routing xử lý request URL ở server và quyết định response. Một SPA production thường cần cả hai: server phải phục vụ app shell cho direct URL, còn client router chọn page sau khi app chạy.

Tình huống	Client router	Server/web host
Click `/orchids/3` từ app	Match route và render detail	Không nhất thiết request document mới
Refresh `/orchids/3`	Chưa chạy cho đến khi HTML/JS tải	Phải trả app shell hoặc SSR page
GET `/api/orchids/3`	Không nên xử lý như page route	Backend API xử lý
Unknown `/abc`	Render route 404	Có thể fallback app shell trước

4. React Router – vai trò và mode

React Router hiện hỗ trợ nhiều mode. Khóa học dùng Declarative routing vì phù hợp mục tiêu BrowserRouter/Routes/Route/Link/hooks. Data/Framework mode có loader/action và nhiều khả năng hơn, nhưng không cần thiết cho Chapter 09 này.

5. BrowserRouter

BrowserRouter đồng bộ React Router với browser History API. Nó phải bao quanh phần app dùng routing context. Trong course track, đặt BrowserRouter ở `main.jsx` giúp `App.jsx` chỉ tập trung route tree.

```
// main.jsx - course track (React Router v7)
import React from "react";
import ReactDOM from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import "bootstrap/dist/css/bootstrap.min.css";
import "./styles/app.css";
import App from "./App.jsx";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>
);
```

6. Routes và Route – route tree, không phải switch-case thủ công

`<Routes>` chọn nhánh route phù hợp với location hiện tại; React Router xếp hạng/match route branches thay vì yêu cầu sinh viên phụ thuộc vào thứ tự “route đầu tiên”. Mỗi `<Route>` định nghĩa pattern và element; route lồng nhau hình thành hierarchy.

```
<Routes>
  <Route path="/" element={<MainLayout />} />
  <Route index element={<HomePage />} />
  <Route path="orchids" element={<OrchidsPage />} />
  <Route path="orchids/:id" element={<OrchidDetailPage />} />
  <Route path="about" element={<AboutPage />} />
  <Route path="*" element={<NotFoundPage />} />
</Routes>
```

7. Link và NavLink

Link dùng cho navigation nội bộ. NavLink bổ sung active/pending state phù hợp menu. Với Home `/`, nên dùng `end` để tránh Home active ở mọi child path.

```
<NavLink to="/" end className={({ isActive }) => isActive ? "nav-link active" : "nav-link"}>Home</NavLink>
<NavLink to="/orchids" className={({ isActive }) => isActive ? "nav-link active" : "nav-link"}>Orchids</NavLink>
```

7.1 Khi nào vẫn dùng `<a>`?

External URL, file download hoặc navigation cần full document request có thể dùng `<a>`. Quy tắc đúng không phải “không bao giờ dùng a”, mà là internal SPA route → Link/NavLink; external/document navigation → anchor phù hợp.

8. useNavigate – navigation theo hành động

Dùng khi navigation là kết quả của event/logic: submit thành công, login, Cancel/Back. Không dùng `useNavigate` thay cho mọi Link vì Link tốt hơn cho semantics/accessibility của navigation thông thường.

```
const navigate = useNavigate();

function handleOpenDetail(id) {
  navigate(`/orchids/${id}`, { state: { from: "/orchids" } });
}

function handleBack() {
  navigate(-1);
}
```

9. Route params và useParams

Dynamic segment `:id` đại diện identity trong path. `useParams()` trả string; nếu ID logic cần number thì phải parse/validate thay vì so sánh lỏng.

```
const { id } = useParams();
const orchid = OrchidsData.find(item => item.id === id);
if (!orchid) return <ResourceNotFound />;
```

10. Query params – nên dùng URL cho filter/search shareable

Prompt yêu cầu query params và useLocation. Ngoài `location.search`, React Router có `useSearchParams`, phù hợp hơn cho việc đọc/ghi query ở Declarative apps. Ví dụ `/orchids?category=Dendrobium&special=true`.

```
const [searchParams, setSearchParams] = useSearchParams();
const category = searchParams.get("category") ?? "all";

function changeCategory(value) {
  const next = new URLSearchParams(searchParams);
  value === "all" ? next.delete("category") : next.set("category", value);
  setSearchParams(next);
}
```

11. useLocation – đọc location object và navigation state

```
const location = useLocation();
console.log(location.pathname); // /orchids/3
console.log(location.search); // ?source=featured
console.log(location.hash); // #care
console.log(location.state); // transient history state
```

Design note: `location.state` không phải dữ liệu bền vững/shareable như URL. Refresh/direct link có thể không có state mong đợi; identity/filter quan trọng nên nằm trong path/query hoặc được tải lại từ data source.

12. Navigate – redirect declarative

```
<Route path="/home" element={<Navigate to="/" replace />} />
```

`replace` không thêm một entry mới vào history; dùng khi alias/redirect không nên làm Back quay lại URL cũ.

13. Nested Routes, Layout, Outlet và index route

Nested route cho phép parent giữ layout còn child thay view. `Outlet` là vị trí render matched child. `index` biểu diễn child mặc định tại đúng parent path.

```
<Route path="dashboard" element={<DashboardLayout />}>
  <Route index element={<DashboardHome />} />
  <Route path="favorites" element={<FavoritesPage />} />
  <Route path="profile" element={<ProfilePage />} />
</Route>

function DashboardLayout() {
  return (
    <Container className="py-4">
      <h2>Dashboard</h2>
      <NavLink to="/dashboard" end>Overview</NavLink>{" | "}
      <NavLink to="/dashboard/favorites">Favorites</NavLink>
      <hr />
      <Outlet />
    </Container>
  );
}
```

14. Route 404 vs Resource Not Found vs API 404

Loại	Ví dụ	AI xử lý
Route not found	/random-path	Router wildcard `**`
Resource not found	/orchids/999 nhưng route pattern match	Page/component/service
API 404	GET /api/orchids/999	Backend/API + frontend error handling

15. Browser history – push, replace, Back/Forward

Link/navigate thường push entry; `replace` thay entry hiện tại; `navigate(-1)` di chuyển history. Sinh viên phải test Back/Forward vì routing đúng không chỉ là render đúng khi click menu.

16. Direct URL, refresh và deployment fallback

Development server thường hỗ trợ SPA fallback. Khi deploy static hosting/web server, request trực tiếp `/orchids/3` phải được rewrite về `index.html` (trừ `/api`/`assets`). Nếu không, internal click chạy nhưng refresh deep link có thể 404 ở server.

17. Route state ownership

State	Nơi phù hợp	Ví dụ
Path param	URL path	/orchids/3
Query	URL search	?category=Vanda
Location state	History entry	from: /orchids
Component state	Component	modal open/closed
Server data	Service/cache	orchid object loaded by id

18. Kỹ năng phải đạt

- Thiết kế route tree trước khi code.
- Dùng đúng Link/NavLink/useNavigate.
- Đọc param/query/location và phân biệt semantics.
- Xây nested layout bằng Outlet.
- Xử lý route 404/resource missing/direct URL.
- Trace history Back/Forward.
- Chuyển component app Lab 02 thành SPA mà không nhồi mọi state vào router.

19. Hiểu sai phổ biến

Hiểu sai	Điều chỉnh
SPA = một màn hình	SPA có nhiều routes/views trong một document shell.
Client-side route không liên quan server	Deep link/refresh vẫn cần server fallback.
Routes render “route đầu tiên”	Router match/rank route branches; không dạy như switch theo thứ tự.
Link và a giống nhau	Internal SPA vs document navigation khác semantics.
useParams = useLocation	params đến từ route pattern; location là URL/history object.
Nested route không cần Outlet	Không có Outlet thì child không có nơi render.
URL state luôn nên giữ trong useState	Shareable navigation/filter state thường nên nằm trong URL.

III. ỨNG DỤNG THỰC TIỄN KIẾN THỨC CỦA BÀI HỌC TRONG SẢN PHẨM THỰC TẾ

1. Route map trong các hệ thống thực tế

Domain	Route ví dụ	Ý nghĩa
E-commerce	/products, /products/:id, /cart	Catalog → detail → transactional view
News	/news, /news/:id, /categories/:id	List/detail/category
LMS	/courses, /courses/:id/lessons/:lessonId	Nested learning hierarchy
Admin	/dashboard/users, /dashboard/settings	Shared layout + child route
Orchid Lab	/orchids, /orchids/:id, /about	Direct bridge Lab 02

2. Case study FUNews – information architecture trước routing code

FUNews có Dashboard, Category, News, Users, Settings. Trước khi viết `<Route>`, nhóm phải quyết định route name, hierarchy, params, query, 404 và quyền truy cập. Router là biểu hiện của information architecture, không phải chỗ “nhét menu”.

View	Route đề xuất	URL state
Dashboard	/dashboard	none
News list	/news?page=1&sort=publishedAt,desc	query
News detail	/news/:id	path param
Category	/categories/:id	path param
Users	/users?role=staff	query
Settings	/settings	component/server state

3. Route trace – từ click đến UI

1. User click NavLink “Orchids”.
2. Router intercepts internal navigation and updates history/location.
3. Route tree matches `/orchids`.
4. MainLayout vẫn tồn tại; Outlet child thay thành OrchidsPage.
5. OrchidsPage render static data hoặc ở slot khác có thể fetch data.
6. Back quay về entry trước mà không tải lại toàn document.

4. Page route và API endpoint – đừng nhầm

Ví dụ: Frontend route `/orchids/3` là địa chỉ UI. Backend endpoint tương lai có thể là `/api/orchids/3`. Router quản lý cái thứ nhất; Fetch/Axios quản lý request đến cái thứ hai.

5. Từ Lab 02 component app sang SPA

Lab 02 gốc tập trung React-Bootstrap, NavBar, Orchids list, Card và Modal. Slot 10 bắt đầu tổ chức những component đó vào route/page mà không bắt buộc thay Modal bằng detail page. Có thể duy trì Modal cho Activity gốc và thêm `/orchids/:id` như extension routing để chuẩn bị Assignment.

IV. NỘI DUNG CHI TIẾT VỀ HƯỚNG DẪN VIỆC THỰC HIỆN VÀ TRIỂN KHAI

A. DEMO TỔNG HỢP – ORCHID ROUTER SPA

Mục tiêu demo: Xây một SPA hoàn chỉnh đủ để trace Router: Home → Orchid list → dynamic detail → query filter → nested dashboard → redirect → 404 → history. Dữ liệu tĩnh để không lặp Slot 9 Fetch/Axios.

A0. Route map trước khi code

Route	Page/Layout	Concept
/	HomePage	index route
/orchids	OrchidsPage	list + query
/orchids/:id	OrchidDetailPage	dynamic param
/about	AboutPage	static route
/contact	ContactPage	useNavigate
/dashboard	DashboardLayout → index	nested/layout
/dashboard/favorites	FavoritesPage	nested child
/home	Navigate → /	redirect
*	NotFoundPage	route 404

Step 1 – Tạo project phù hợp Node 21

```
npm create vite@5 orchid-router-demo -- --template react
cd orchid-router-demo
npm install
npm run dev
```

Kết quả: Vite dev server chạy; ghi lại URL thực tế (thường 5173 nhưng có thể đổi nếu port bận).

Step 2 – Cài React Router + React Bootstrap

```
npm install react-router-dom@7.18.0 react-bootstrap bootstrap
```

Kiểm tra `package.json`. Không dùng `@latest` trên Node 21 vì current Router/Vite baseline đã thay đổi.

Step 3 – Tạo cấu trúc thư mục

```
src/  
├── components/  
│   ├── AppNavbar.jsx  
│   └── OrchidCard.jsx  
├── data/  
│   └── orchids.js  
├── layouts/  
│   ├── MainLayout.jsx  
│   └── DashboardLayout.jsx  
├── pages/  
│   ├── HomePage.jsx  
│   ├── OrchidsPage.jsx  
│   ├── OrchidDetailPage.jsx  
│   ├── AboutPage.jsx  
│   ├── ContactPage.jsx  
│   ├── DashboardHome.jsx  
│   ├── FavoritesPage.jsx  
│   ├── ProfilePage.jsx  
│   └── NotFoundPage.jsx  
├── styles/app.css  
├── App.jsx  
└── main.jsx
```

Mỗi file có một trách nhiệm rõ ràng; tránh cấu trúc “StaticPages.jsx chứa nhiều component” vì khó trace khi sinh viên mới học route-to-file mapping.

Step 4 – Tạo dữ liệu Orchids

```
// src/data/orchids.js
export const OrchidsData = [
  { id: "1", orchidName: "Ceasar 4N", category: "Dendrobium", isSpecial: true,
    image: "https://images.unsplash.com/photo-1566907225472-7e50f33ba1a6?auto=format&fit=crop&w=900&q=80",
    description: "Dendrobium hybrid with a strong visual form." },
  { id: "2", orchidName: "Mini Yaya", category: "Dendrobium", isSpecial: true,
    image: "https://images.unsplash.com/photo-1567410259730-7c6f9c7912b9?auto=format&fit=crop&w=900&q=80",
    description: "Compact orchid suitable for a small display." },
  { id: "3", orchidName: "Crystallinum", category: "Dendrobium", isSpecial: false,
    image: "https://images.unsplash.com/photo-1557968581-06d4b39ec2d7?auto=format&fit=crop&w=900&q=80",
    description: "An orchid entry used for routing practice." },
  { id: "4", orchidName: "Taichung Beauty", category: "Cattleya", isSpecial: true,
    image: "https://images.unsplash.com/photo-1494336934272-f44515c5f5b7?auto=format&fit=crop&w=900&q=80",
    description: "A Cattleya example for category filtering." },
  { id: "5", orchidName: "Blue Vanda", category: "Vanda", isSpecial: false,
    image: "https://images.unsplash.com/photo-1606041008023-472dfb5e530f?auto=format&fit=crop&w=900&q=80",
    description: "Vanda example for query-parameter practice." },
  { id: "6", orchidName: "Moon Phalaenopsis", category: "Phalaenopsis", isSpecial: false,
    image: "https://images.unsplash.com/photo-1602422086108-8e7d9b2c9a1f?auto=format&fit=crop&w=900&q=80",
    description: "Phalaenopsis example for a detail route." },
];
```

Asset note: Ảnh web chỉ là dữ liệu demo. Với Lab 02 chính thức nên dùng ảnh hợp lệ do giảng viên cung cấp hoặc asset local để tránh phụ thuộc URL ngoài.

Step 5 – main.jsx: BrowserRouter ở app root

```
import React from "react";
import ReactDOM from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import "bootstrap/dist/css/bootstrap.min.css";
import "./styles/app.css";
import App from "./App.jsx";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>
);
```

Step 6 – AppNavbar.jsx: NavLink và active state

```
import { Container, Nav, Navbar } from "react-bootstrap";
import { NavLink } from "react-router-dom";

const navClass = ({ isActive }) =>
  isActive ? "nav-link active fw-semibold" : "nav-link";

export default function AppNavbar() {
  return (
    <Navbar bg="primary" data-bs-theme="dark" expand="lg">
      <Container>
        <Navbar.Brand as={NavLink} to="/">Orchid Gallery</Navbar.Brand>
        <Navbar.Toggle aria-controls="main-nav" />
        <Navbar.Collapse id="main-nav">
          <Nav className="me-auto">
            <Nav.Link as={NavLink} to="/" end className={navClass}>Home</Nav.Link>
            <Nav.Link as={NavLink} to="/orchids" className={navClass}>Orchids</Nav.Link>
            <Nav.Link as={NavLink} to="/dashboard" className={navClass}>Dashboard</Nav.Link>
            <Nav.Link as={NavLink} to="/about" className={navClass}>About</Nav.Link>
            <Nav.Link as={NavLink} to="/contact" className={navClass}>Contact</Nav.Link>
          </Nav>
        </Navbar.Collapse>
      </Container>
    </Navbar>
  );
}
```

Step 7 – MainLayout.jsx: layout + Outlet

```
import { Outlet } from "react-router-dom";
import AppNavbar from "../components/AppNavbar";

export default function MainLayout() {
  return (
    <>
      <AppNavbar />
      <main><Outlet /></main>
      <footer className="border-top py-3 text-center text-muted">
        SBA301 • Orchid Router Demo
      </footer>
    </>
  );
}
```

Step 8 – Static pages tách file riêng

```
// HomePage.jsx
import { Container } from "react-bootstrap";
export default function HomePage() {
  return <Container className="py-5"><h1>Orchid Gallery SPA</h1><p>Explore orchids through URL-driven navigation.</p></Container>;
}

// AboutPage.jsx
import { Container } from "react-bootstrap";
export default function AboutPage() {
  return <Container className="py-5"><h1>About</h1><p>Slot 10 demo for React Router and SPA.</p></Container>;
}
```

Step 9 – OrchidCard.jsx: Link tới dynamic route

```
import { Button, Card } from "react-bootstrap";
import { Link } from "react-router-dom";

export default function OrchidCard({ orchid }) {
  return (
    <Card className="h-100 shadow-sm">
      <Card.Img className="orchid-image" variant="top" src={orchid.image} alt={orchid.orchidName} />
      <Card.Body className="d-flex flex-column">
        <Card.Title>{orchid.orchidName}</Card.Title>
        <Card.Text>{orchid.category}</Card.Text>
        <Button as={Link} to={` /orchids/${orchid.id}`} state={{ from: "/orchids" }} className="mt-auto">
          View Detail
        </Button>
      </Card.Body>
    </Card>
  );
}
```

Step 10 – OrchidsPage.jsx: query state với useSearchParams

```
import { Col, Container, Form, Row } from "react-bootstrap";
import { useSearchParams } from "react-router-dom";
import { OrchidsData } from "../data/orchids";
import OrchidCard from "../components/OrchidCard";

export default function OrchidsPage() {
  const [searchParams, setSearchParams] = useSearchParams();
  const category = searchParams.get("category") ?? "all";

  const categories = ["all", ...new Set(OrchidsData.map(o => o.category))];
  const visible = category === "all"
    ? OrchidsData
    : OrchidsData.filter(o => o.category === category);

  function changeCategory(event) {
    const value = event.target.value;
    const next = new URLSearchParams(searchParams);
    value === "all" ? next.delete("category") : next.set("category", value);
    setSearchParams(next);
  }

  return (
    <Container className="py-4">
      <div className="d-flex flex-wrap justify-content-between align-items-center gap-2 mb-3">
        <h1 className="mb-0">Orchids</h1>
        <Form.Select value={category} onChange={changeCategory} style={{ maxWidth: 260 }}>
          {categories.map(item => <option key={item} value={item}>{item}</option>)}
        </Form.Select>
      </div>
      <p className="text-muted">Current filter is encoded in the URL query string.</p>
      <Row xs={1} md={2} lg={3} className="g-4">
        {visible.map(orchid => <Col key={orchid.id}><OrchidCard orchid={orchid} /></Col>)}
      </Row>
    </Container>
  );
}
```

Step 11 – OrchidDetailPage.jsx: useParams + useNavigate + useLocation

```
import { Button, Container } from "react-bootstrap";
import { Link, useLocation, useNavigate, useParams } from "react-router-dom";
import { OrchidsData } from "../data/orchids";

export default function OrchidDetailPage() {
  const { id } = useParams();
  const location = useLocation();
  const navigate = useNavigate();
  const orchid = OrchidsData.find(item => item.id === id);

  if (!orchid) {
    return (
      <Container className="py-5">
        <h1>Orchid not found</h1>
        <p>The route exists, but resource ID <code>{id}</code> does not.</p>
        <Button as={Link} to="/orchids">Back to Orchids</Button>
      </Container>
    );
  }

  return (
    <Container className="py-4">
      <Button variant="outline-secondary" onClick={() => navigate(-1)}>< Browser Back</Button>
      <h1 className="mt-3">{orchid.orchidName}</h1>
      <img src={orchid.image} alt={orchid.orchidName} className="detail-image mb-3" />
      <p><strong>Category:</strong> {orchid.category}</p>
      <p><strong>Special:</strong> {orchid.isSpecial ? "Yes" : "No"}</p>
      <p>{orchid.description}</p>
      <small className="text-muted">Navigation source: {location.state?.from ?? "direct URL / refresh"}</small>
    </Container>
  );
}
```

Step 12 – ContactPage.jsx: programmatic navigation

```
import { Button, Container } from "react-bootstrap";
import { useNavigate } from "react-router-dom";

export default function ContactPage() {
  const navigate = useNavigate();

  function handleSubmit(event) {
    event.preventDefault();
    // Demo only: no backend submission in Slot 10.
    navigate("/", { replace: true, state: { message: "Contact form completed" } });
  }

  return (
    <Container className="py-5">
      <h1>Contact</h1>
      <form onSubmit={handleSubmit}>
        <Button type="submit">Submit demo form</Button>
      </form>
    </Container>
  );
}
```

Step 13 – DashboardLayout + nested pages

```
// DashboardLayout.jsx
import { Container, Nav } from "react-bootstrap";
import { NavLink, Outlet } from "react-router-dom";

export default function DashboardLayout() {
  return (
    <Container className="py-4">
      <h1>Dashboard</h1>
      <Nav variant="tabs" className="mb-3">
        <Nav.Link as={NavLink} to="/dashboard" end>Overview</Nav.Link>
        <Nav.Link as={NavLink} to="/dashboard/favorites">Favorites</Nav.Link>
        <Nav.Link as={NavLink} to="/dashboard/profile">Profile</Nav.Link>
      </Nav>
      <Outlet />
    </Container>
  );
}

// DashboardHome.jsx
export default function DashboardHome() { return <p>Dashboard overview.</p>; }

// FavoritesPage.jsx
export default function FavoritesPage() { return <p>Favorite orchids will appear here.</p>; }

// ProfilePage.jsx
export default function ProfilePage() { return <p>Profile settings placeholder.</p>; }
```

Step 14 – NotFoundPage.jsx

```
import { Button, Container } from "react-bootstrap";
import { Link, useLocation } from "react-router-dom";

export default function NotFoundPage() {
  const location = useLocation();
  return (
    <Container className="py-5 text-center">
      <h1>404 – Route Not Found</h1>
      <p>No page matches <code>{location.pathname}</code>.</p>
      <Button as={Link} to="/">Go Home</Button>
    </Container>
  );
}
```

Step 15 – App.jsx: route tree hoàn chỉnh

```
import { Navigate, Route, Routes } from "react-router-dom";
import MainLayout from "../layouts/MainLayout";
import DashboardLayout from "../layouts/DashboardLayout";
import HomePage from "../pages/HomePage";
import OrchidsPage from "../pages/OrchidsPage";
import OrchidDetailPage from "../pages/OrchidDetailPage";
import AboutPage from "../pages/AboutPage";
import ContactPage from "../pages/ContactPage";
import DashboardHome from "../pages/DashboardHome";
import FavoritesPage from "../pages/FavoritesPage";
import ProfilePage from "../pages/ProfilePage";
import NotFoundPage from "../pages/NotFoundPage";

export default function App() {
  return (
    <Routes>
      <Route path="/" element={<MainLayout />} />
      <Route index element={<HomePage />} />
      <Route path="orchids" element={<OrchidsPage />} />
      <Route path="orchids/:id" element={<OrchidDetailPage />} />
      <Route path="about" element={<AboutPage />} />
      <Route path="contact" element={<ContactPage />} />
      <Route path="home" element={<Navigate to="/" replace />} />

      <Route path="dashboard" element={<DashboardLayout />} />
      <Route index element={<DashboardHome />} />
      <Route path="favorites" element={<FavoritesPage />} />
      <Route path="profile" element={<ProfilePage />} />
    </Route>

    <Route path="*" element={<NotFoundPage />} />
  </Route>
</Routes>
);
}
```

Step 16 – CSS tối thiểu

```
.orchid-image {
  height: 220px;
  object-fit: cover;
}

.detail-image {
  width: min(100%, 720px);
  max-height: 460px;
  object-fit: cover;
  border-radius: 12px;
}

.nav-link.active {
  font-weight: 700;
}
```

Step 17 – Route verification matrix

Test	Thao tác	Expected
R01	Open `/`	Home + navbar + footer
R02	Click Orchids	URL `/orchids`, no full reload
R03	Choose Vanda	URL contains `?category=Vanda`
R04	Open detail ID 2	`/orchids/2`, correct orchid
R05	Open ID 999	Resource-not-found message
R06	Open Dashboard	Index child visible
R07	Open Favorites	Nested Outlet changes
R08	Open `/home`	Redirect to `/`, no extra Back trap

Test	Thao tác	Expected
R09	Open `/xyz`	Route 404
R10	Back/Forward	History traverses views
R11	Refresh `/orchids/2`	Works in dev; note deployment fallback
R12	Open direct detail new tab	Detail loads without location.state dependency

Step 18 – DevTools / history observation

1. Open DevTools → Network, enable Preserve log if needed.
2. Navigate Home → Orchids → Detail. Verify no new document request for each internal navigation.
3. Use Back/Forward; record URL and component shown.
4. Open direct `/orchids/2`; verify page works even without `location.state`.
5. Compare internal `Link` with temporary `

Step 19 – Git checkpoint

```
git status
git add .
git commit -m "slot10: build orchid SPA routing demo"
```

B. ROUTE TRACE WORKSHEET – BẮT BUỘC

Navigation	Old URL	Action	New URL	Matched route	Rendered Outlet/Page	History effect
Home → Orchids	/	NavLink	/orchids	orchids	OrchidsPage	push
Orchids → Detail	/orchids	Link	/orchids/2	orchids/:id	OrchidDetailPage	push
Detail → Back	/orchids/2	navigate(-1)	/orchids	orchids	OrchidsPage	traverse
Alias home	/home	Navigate replace	/	index	HomePage	replace

C. BREAK-IT LAB – HỌC ROUTER QUA LỖI

Bug injection	Triệu chứng	Nguyên nhân cần giải thích	Fix
Bỏ BrowserRouter	Hook/components lỗi context	Router context thiếu	Wrap root
Dùng ` <a>` cho `/orchids`	Document reload	Browser navigation mặc định	Link/NavLink
Bỏ `Outlet`	Child URL đổi nhưng UI không đổi	Không có render slot	Add Outlet
Route detail thành `/orchid/:id` nhưng Link vẫn `/orchids/:id`	404 route	Path contract mismatch	Unify route
Dùng `id === 2`	Resource not found	useParams returns string	Compare/parse correctly
Home NavLink không `end`	Home active cả child route	Prefix match	Add end
Chỉ dựa location.state cho orchid	Direct URL fail	Transient state not durable	Load/find by URL id
Bỏ wildcard route	Blank/unexpected no match	No fallback route	Add `*`

D. DEBUG DECISION TREE

1. URL không đổi → kiểm tra element có dùng Link/NavLink/useNavigate đúng không.
2. URL đổi nhưng page không đổi → kiểm tra route pattern/nested Outlet.

3. Page detail sai dữ liệu → inspect params type/value và data lookup.
4. Active menu sai → kiểm tra NavLink, `end`, nested path.
5. Back/Forward sai → kiểm tra `replace` vs push và programmatic navigation.
6. Internal click chạy nhưng refresh 404 → vấn đề deployment/server fallback, không phải route matching phía client.
7. Direct URL chỉ fail khi thiếu `location.state` → state ownership sai; dữ liệu quan trọng phải reconstruct được từ URL/data source.

E. BẮT ĐẦU LAB 02 – CREATE SINGLE PAGE APPLICATION WITH REACTJS

E1. Mục tiêu bridge

Lab 02 gốc yêu cầu React-Bootstrap, NavBar, Orchid list, Card và Modal. Slot 10 bắt đầu Lab bằng việc đảm bảo project/UI foundation đúng, sau đó route hóa những phần phù hợp. Không bắt buộc bỏ Modal detail nếu rubric Lab 02 vẫn yêu cầu Modal.

E2. Activity 01 – Consume React-Bootstrap

```
npm install react-bootstrap bootstrap
// main.jsx
import "bootstrap/dist/css/bootstrap.min.css";
```

- Tạo/reuse NavBar component.
- Nếu đã dùng AppNavbar ở demo, map lại các menu theo yêu cầu Lab 02.
- Không duplicate BrowserRouter ở cả main.jsx và App.jsx.

E3. Activity 02 – Design List of Orchids

1. Đưa dữ liệu Orchid về `src/shared/ListOfOrchids.js` hoặc `src/data/orchids.js` theo yêu cầu lớp.
2. Tạo/reuse `OrchidCard`.
3. Tạo `Orchids` component/page hiển thị Card grid.
4. Giữ Modal detail nếu Lab 02 yêu cầu; route detail là extension.
5. Kiểm tra responsive layout bằng React Bootstrap.

E4. Lab 02 readiness checklist

Check	Đạt khi...
Bootstrap	CSS import đúng, NavBar/Card/Button có style
Navbar	Internal navigation không full reload
Orchids list	Card có image/name/category/detail
Modal	selectedOrchid + show/close hoạt động nếu yêu cầu
Router	/orchids và ít nhất một route khác chạy
Dynamic route	Extension `/orchids/:id` đọc đúng param
Code structure	component/layout/page tách rõ

V. KINH NGHIỆM THỰC TIỄN, TIPS VÀ BEST PRACTICES

1. Thiết kế route trước khi viết code

Một route table 5 phút thường tiết kiệm nhiều thời gian sửa link/path về sau. Với mỗi route, ghi rõ page, params, query, parent layout, auth requirement và expected 404 behavior.

2. URL là API cho người dùng

- Tên path ngắn, có nghĩa và ổn định.
- Identity → path params; filter/shareable state → query.
- Không đưa object lớn/nặng cảm vào URL.
- Không phụ thuộc location.state cho deep-link critical data.

3. Link vs useNavigate

Use case	Ưu tiên
Menu/text navigation	Link/NavLink
Sau submit/login	useNavigate hoặc data-router redirect ở kiến trúc nâng cao
Back button	navigate(-1) nếu semantics thực sự là browser back
Alias cũ	Navigate replace

4. 404 có ba lớp

Route 404, resource-not-found và backend API 404 phải có UI/message và debug path khác nhau. Trộn chúng khiến sinh viên khó chẩn đoán “router sai hay data sai”.

5. Nested route chỉ nên dùng khi hierarchy thật sự có ý nghĩa

Không nested chỉ để “code nhìn chuyên nghiệp”. Nested phù hợp khi child chia sẻ layout/context/navigation và URL hierarchy có quan hệ cha-con.

6. Deep link là quality gate

Mọi route quan trọng phải mở được bằng cách paste URL vào tab mới. Đây là kiểm tra đơn giản nhưng phát hiện location-state coupling và deploy fallback rất tốt.

7. Accessibility

- Navigation nên dùng semantic links, không biến mọi thứ thành button + navigate.
- NavLink active state cần đủ rõ, không chỉ dựa màu nếu có thể.
- 404 page phải có đường quay lại.
- Focus/heading của page nên hợp lý khi route thay đổi.

8. Version note

Syllabus/slide có thể dùng syntax React Router v6. Course track pin v7.18 để giữ declarative API tương thích. Khi nâng lên v8, kiểm tra package/import baseline trước; không “npm install latest” máy móc trên Node 21.

VI. CÁC BÀI TẬP CHỦ ĐỀ VÀ BÀI TẬP TỔNG HỢP

A. 10 bài tập chủ đề

#	Bài tập	Yêu cầu chứng minh
1	SPA vs MPA trace	Vẽ 2 flow navigation và chỉ ra document request.
2	Basic routes	Home/About/Contact + wildcard.
3	NavLink active	Home dùng end; chụp 2 URL.
4	Dynamic route	/products/:id + invalid ID.
5	Query params	Filter `/orchids?category=...`.
6	useNavigate	Submit demo → redirect replace.
7	useLocation	Hiển thị pathname/search/state.
8	Nested routes	Dashboard + index + 2 children.

#	Bài tập	Yêu cầu chứng minh
9	History	Test push/replace/Back/Forward.
10	Deep link	Paste route URL vào tab mới và phân tích.

B. Bài tập tổng hợp – Orchid SPA with React Router

Sinh viên xây Orchid SPA dựa trên dữ liệu Lab 02, nhưng không đi sâu Fetch/Axios. Sản phẩm phải có route map, navigation, list, dynamic detail, query filter, nested dashboard, redirect, route 404 và evidence.

B1. Functional requirements

1. FR01 `/` Home.
2. FR02 `/orchids` list cards.
3. FR03 category filter lưu trong query.
4. FR04 `/orchids/:id` detail.
5. FR05 invalid ID có resource-not-found.
6. FR06 `/about`.
7. FR07 `/dashboard/\*` nested route.
8. FR08 `/home` redirect `/`.
9. FR09 wildcard route 404.
10. FR10 Back/Forward và direct URL chạy.

B2. Quality requirements

- Không dùng full reload cho internal navigation.
- Không phụ thuộc transient location state cho page identity.
- Tách layout/page/component/data.
- Không có console error nghiêm trọng.
- README có route table và run steps.

B3. Verification matrix

ID	Scenario	Evidence
T01	Home → Orchids	URL + active menu screenshot
T02	Category filter	query string screenshot
T03	Detail valid	URL + page
T04	Detail invalid	resource-not-found
T05	Unknown route	404 page
T06	Dashboard child	Outlet output
T07	Back/Forward	history trace
T08	Direct URL	new-tab proof
T09	Internal link	Network: no document reload
T10	Anchor experiment	Network: document reload observed

C. Mini Design Review

1. Tại sao category filter là query chứ không useState-only?
2. Tại sao Orchid ID là path param?
3. Tại sao invalid ID không nên dùng route `\*`?
4. Tại sao MainLayout cần Outlet?
5. Tại sao `location.state` chỉ là bổ sung?
6. Nếu deploy lên server thật, cần rewrite gì cho deep link?

VII. TỪ ĐIỂN THUẬT NGỮ – FOUNDATION GLOSSARY

Thuật ngữ	Định nghĩa	Ví dụ
SPA	Single Page Application: nhiều views trong một document shell.	/orchids → /about không full reload
MPA	Navigation thường tải document/page mới từ server.	Server-rendered pages
Client-side routing	Frontend runtime map URL sang view.	React Router
Server-side routing	Server map request URL sang response.	GET /api/... hoặc HTML route
React Router	Routing library cho React.	Declarative route tree
BrowserRouter	Router dựa trên browser History API.	Wrap App root
Routes	Container render matched route branch.	<Routes>...</Routes>
Route	Mapping giữa pattern và element.	path="/orchids/:id"
Link	Semantic internal navigation.	to="/about"
NavLink	Link có active/pending state.	Main menu
useNavigate	Programmatic navigation hook.	navigate(-1)
useParams	Đọc dynamic path params.	id từ /orchids/:id
useLocation	Đọc location object.	pathname/search/state
useSearchParams	Đọc/ghi query string.	?category=Vanda
Navigate	Declarative redirect/navigation.	/home → /
Nested Route	Child route dưới parent route.	dashboard/favorites
Outlet	Placeholder render matched child.	DashboardLayout
Index Route	Default child ở parent URL.	/dashboard overview
Route Param	Dynamic path segment.	/:id
Query Param	Key/value trong search string.	?page=2
Location State	State gắn history entry.	{from:"/orchids"}
History API	Browser navigation stack API.	push/replace/back
Push	Thêm history entry.	Link default
Replace	Thay current history entry.	Navigate replace
Deep Link	URL trực tiếp tới nested/detail view.	/orchids/3
Route 404	Không có route pattern match.	path="*"
Resource 404	Route match nhưng data ID không tồn tại.	/orchids/999
Active State	Trạng thái link khớp current location.	NavLink active
Layout Route	Parent route giữ shared UI.	MainLayout
SPA Fallback	Server rewrite unknown page route về index.html.	Refresh /orchids/3

VIII. RESEARCH BASE VÀ AI BASE LEARNING

A. 10 prompt tự nghiên cứu chuyên sâu

1. Phân tích SPA và MPA ở cấp document navigation, rendering và data flow; đưa ví dụ không tuyệt đối hóa ưu nhược điểm.

2. Giải thích HTML5 History API được BrowserRouter sử dụng như thế nào và Back/Forward khác push/replace ra sao.
3. Phân tích route ranking/matching trong React Router và vì sao không nên mô tả Routes như “render route đầu tiên theo thứ tự”.
4. Giải thích nested routes, layout routes, index routes và Outlet bằng một route tree cụ thể.
5. Phân tích sự khác nhau giữa path params, query params, location state và component state theo tiêu chí shareable/persistent.
6. Giải thích khi nào dùng Link/NavLink và khi nào dùng useNavigate; đưa anti-pattern.
7. Phân tích route 404, resource-not-found và API 404; đề xuất test cho từng loại.
8. Giải thích direct URL/refresh trong SPA và cách web server history fallback hoạt động.
9. Phân tích cách route-based code splitting/lazy loading có thể cải thiện initial bundle (chỉ ở mức research).
10. So sánh Declarative, Data và Framework modes của React Router hiện đại; xác định mode phù hợp khóa học SBA301.

B. 10 prompt nghiên cứu production practice

1. Thiết kế route convention cho admin system 50 routes, gồm naming, nesting, 404 và breadcrumb strategy.
2. Làm thế nào để migrate route `/news/:id` sang `/articles/:slug` mà hạn chế breaking bookmarks?
3. Trong production, nên xử lý SPA fallback ở Nginx/Apache/static host như thế nào và tránh rewrite nhầm `/api`?
4. Thiết kế protected-route strategy nhưng tránh lưu authorization chỉ ở client router.
5. Khi nào filter/search nên ở query string thay vì local state? Đưa decision table.
6. Đề xuất testing strategy cho navigation: unit/component/e2e và direct URL.
7. Làm thế nào để instrument route changes cho analytics mà không làm rối component code?
8. Thiết kế breadcrumbs từ nested route metadata cho hệ thống lớn.
9. Phân tích accessibility khi SPA navigation thay đổi page content nhưng browser không reload document.
10. Đề xuất route ownership theo feature module trong team nhiều developer để giảm conflict.

C. 10 prompt khám phá công nghệ/xu hướng

1. So sánh React Router v8 với v7 về package baseline và import strategy, dùng tài liệu chính thức mới nhất.
2. So sánh React Router với TanStack Router về declarative route, type safety và data loading.
3. So sánh React Router SPA với Next.js App Router ở routing/rendering/data boundaries.
4. File-based routing là gì và trade-off so với route config thủ công?
5. Server Components ảnh hưởng gì đến cách nghĩ client-side routing?
6. Type-safe routing trong TypeScript được triển khai theo những cách nào?
7. View Transitions API có thể hỗ trợ page transition trong router như thế nào?
8. Route-based prefetch/prefetch-on-intent là gì và ảnh hưởng UX ra sao?
9. Micro-frontend routing có các vấn đề ownership/history/base path nào?
10. AI coding assistant có thể tạo route code nhưng cần verification nào để tránh route conflict/deep-link bugs?

D. AI Verification Worksheet

Trưởng	Sinh viên ghi
Prompt	Câu lệnh gửi AI

Trường	Sinh viên ghi
Claim/code AI đưa	Tóm tắt
Official/source check	React Router docs / browser behavior / run result
Experiment	URL/action đã thử
Evidence	Screenshot/console/network
Conclusion	Accept / modify / reject + lý do

AI-0 rule: Không chấp nhận “AI nói route này đúng”. Sinh viên phải mở URL, test Back/Forward/direct URL và giải thích route tree bằng lời của mình.

IX. SINH VIÊN CẦN TỰ KIỂM TRA GÌ SAU BUỔI HỌC

1. Knowledge self-check

1. SPA khác MPA ở mức document navigation như thế nào?
2. Client-side routing và server-side routing phối hợp khi refresh deep link ra sao?
3. BrowserRouter có vai trò gì?
4. Routes/Route tạo route tree như thế nào?
5. Link/NavLink khác `<a>` ở đâu?
6. Khi nào dùng useNavigate?
7. useParams trả dữ liệu kiểu gì?
8. Query nên dùng useSearchParams khi nào?
9. useLocation.state có hạn chế gì?
10. Outlet cần thiết khi nào?
11. Index route là gì?
12. Route 404 khác resource-not-found ra sao?
13. push vs replace ảnh hưởng Back thế nào?
14. Tại sao `/orchids/3` phải chạy được khi paste vào tab mới?

2. Skill checklist

Nội dung kiểm tra	Đạt	Chưa đạt	Cần làm lại
Tạo project compatible Node 21			
Cài/pin React Router			
BrowserRouter root			
Routes + Route tree			
Link/NavLink + active			
useNavigate			

Nội dung kiểm tra	Đạt	Chưa đạt	Cần làm lại
useParams			
useLocation			
useSearchParams			
Nested route + Outlet			
Navigate redirect			
Route 404			
Resource-not-found			
Back/Forward			
Direct URL/deep link			
Lab 02 Activity 01 ready			
Lab 02 Activity 02 ready			

3. Mức độ tự đánh giá

Mức	Biểu hiện
Mức 1 – Chưa đạt	Chỉ copy Route/Link; không trace URL/history; nested/deep link fail.
Mức 2 – Đạt cơ bản	Basic routes, navigation, dynamic param, 404 chạy và giải thích được.
Mức 3 – Tốt	Thiết kế route tree hợp lý; query/nested/history/direct URL đúng; debug được routing bug; chuyển được sang Lab 02.

4. Exit Ticket

1. Một điều về URL/router hôm nay làm thay đổi cách em nghĩ về SPA.
2. Một lỗi routing em tự tạo và tự sửa.
3. Một state em quyết định đưa vào URL thay vì useState, và lý do.
4. Một câu hỏi em còn chưa chắc về nested/deep link.
5. Commit hash cuối buổi.

5. Human Verification Gate – 20 câu

1. Vẽ route tree của app trong 60 giây.
2. Tại sao BrowserRouter đặt ở main.jsx?
3. Nếu bỏ Outlet, URL child có thể đổi không và UI ra sao?
4. Tại sao Home NavLink nên có `end`?
5. `useParams().id` là string hay number?
6. Tại sao category filter dùng query string?
7. Khi nào location.state hữu ích?
8. Vì sao không dựa location.state để load detail?

9. Khác nhau giữa Link và useNavigate?
10. navigate(-1) khác navigate("/orchids") thế nào?
11. replace ảnh hưởng history thế nào?
12. Route 404 khác orchid ID 999 thế nào?
13. Tại sao internal Link không tạo document request mới?
14. Tại sao `<a href>` có thể reload document?
15. Refresh `/orchids/2` phụ thuộc server điều gì?
16. Nếu production refresh 404 nhưng click nội bộ chạy, debug ở frontend hay hosting trước?
17. Query param có ưu điểm gì cho bookmark/share?
18. Nested route phù hợp khi nào?
19. Nếu đổi route path, những file/contracts nào phải kiểm tra?
20. Mở code và chỉ trong 30 giây nơi định nghĩa `/orchids/:id` và nơi đọc `id`.

6. Việc cần làm nếu chưa đạt

- Làm lại Route Trace Worksheet thay vì đọc lại code thụ động.
- Break một route có chủ đích rồi dùng URL/console/router tree để sửa.
- Mở DevTools Network so sánh Link với anchor.
- Paste từng deep link vào tab mới.
- Nhờ AI giải thích nhưng bắt buộc kiểm chứng bằng behavior thật.

PHỤ LỤC A – KẾ HOẠCH GIẢNG 135 PHÚT

Thời gian	Hoạt động	Đầu ra
0–10	Retrieval Slot 9 + pre-check	6–8 câu nhanh
10–30	SPA/MPA + routing mental model	URL → Router diagram
30–50	BrowserRouter/Routes/Link/NavLink	Basic route demo
50–70	Params/query/location/navigate	Route trace
70–90	Nested routes/Outlet/404/history	Dashboard demo
90–110	Orchid Router integrated demo	Working SPA
110–122	Break-It Lab	Bug log
122–132	Lab 02 kick-off	Bootstrap + Orchids plan
132–135	Exit ticket	Evidence + next action

PHỤ LỤC B – QUICK REFERENCE

API	Mục đích	Ví dụ
BrowserRouter	Browser history context	<BrowserRouter>
Routes	Render matched branch	<Routes>
Route	Pattern → element	path="/orchids/:id"
Link	Internal navigation	to="/about"
NavLink	Active navigation	end / isActive
useNavigate	Programmatic navigation	navigate(-1)
useParams	Path params	{ id }
useLocation	Current location/state	pathname/search/state
useSearchParams	Query string state	category=Vanda
Navigate	Declarative redirect	replace
Outlet	Nested child slot	<Outlet />
index	Default child	<Route index ... />
*	Catch-all route	NotFoundPage

PHỤ LỤC C – EVIDENCE PACKAGE E01–E14

Evidence	Nội dung
E01	node/npm + package.json versions
E02	Route map trước khi code
E03	Home + NavLink active
E04	Orchid list route

Evidence	Nội dung
E05	Query filter URL
E06	Dynamic detail valid
E07	Resource-not-found invalid ID
E08	Nested dashboard
E09	Redirect /home
E10	Route 404
E11	Back/Forward trace
E12	Direct URL proof
E13	Break-It bug + fix
E14	Git commit + reflection

PHỤ LỤC D – README/PROJECT SUBMISSION TEMPLATE

```
# Orchid Router Demo - SBA301 Slot 10

## Environment
- Node: ...
- npm: ...
- React Router: ...

## Run
npm install
npm run dev

## Route Map
| URL | Page | Notes |
| --- | --- | --- |
| / | Home | index |
| /orchids | Orchids | query filter |
| /orchids/:id | Detail | route param |

## Verification
- Back/Forward: PASS/FAIL
- Direct URL: PASS/FAIL
- 404: PASS/FAIL
- Resource not found: PASS/FAIL

## Known Limitations
- No Spring Boot backend in Slot 10.
- Fetch/Axios not the focus of this slot.

## Reflection
1. ...
2. ...
```

PHỤ LỤC E – TÀI LIỆU THAM KHẢO VÀ CẬP NHẬT KỸ THUẬT

- Slide Chapter 09 trang 1–22 (nguồn nội dung khóa học).
- React Router official docs: <https://reactrouter.com/>
- React Router Declarative Installation: <https://reactrouter.com/start/declarative/installation>
- React Router v8 upgrade/current notes: <https://reactrouter.com/upgrading/v8> hoặc changelog.
- Vite Getting Started: <https://vite.dev/guide/>
- MDN History API / URL concepts.
- Tài liệu Slot 9 – Fetching and Caching Data.
- Lab 02 – Create Single Page Application with ReactJS.

Technical snapshot: Tại thời điểm biên soạn 20/09/2026, React Router v8.4.0 là release hiện hành. Course track chủ động pin v7.18.0 vì Node 21 và vì slide/lab dùng `react-router-dom`; nội dung concept được viết để chuyển được sang v8 khi môi trường nâng cấp.